

Agenda:

Program Flow Control in Python

if, for

Strings

Slicing

List

Dictionary

while

dict

tuples, sets

functions & Modules

Data Structures in python

Strings

Lists

Dicts

Tuples

Sets

Stacks

Queues

Strings:

Sequence of Characters

It is immutable (cant be modified)

represent a string using '', "", ''' '''

Used to represent text

Each character can be accessed by its
position

posistion(index) starts from 0

Always from left to right

except when step values is -ve

```
v = "vishwas"
print("my name is: "+ 'v')
```

```
name = "vishwas"
print("my name is: "+ "name")
```

```
num = float(input("Enter the number: "))
```

```
option A: 123      option B: '123'
```

```
num / 10
```

```
>>> v = "vishwas"
>>> type(v)
<class 'str'>
>>> v1 = 'vishwas'
>>> type(v1)
<class 'str'>
>>> v2 = '''vishwas'''
>>> type(v2)
<class 'str'>
>>> num = input("Enter a number: ")
Enter a number: 123
>>> num
'123'
>>> type(num)
<class 'str'>
>>> num = float(input("Enter a number: "))
Enter a number: 123
>>> num
123.0
>>> type(num)
<class 'float'>
>>>
```

=.

Vishwas

0 1 2 3 4 5 6

① 0 → V

1 → i

6 → S

② -1 → S

-2 → a

~~-7~~ → V

1: ④

↓

① 2 3

i S h

1+0 → 1 → i

1+1 → 2 → S

1+1+1 → 3 → h

h -1 → ③ Stop

by default Step val

VISHWAS

0

6

↑

← -1

SAWHS/V

-1

[] 0:6: -1

↑

0 → V

0-1 →

1 1

0-1

↓

→

6 5

SA

List
 multiple values in a variable
 Similar to a string → indexing, slicing
 Lists are mutable (can be modified)
 its a heterogenous data structure
 it maintains insertion order
 It can contain duplicates

l5 = l3 they point to same object
 l6 = l3.copy() seperate copy

```

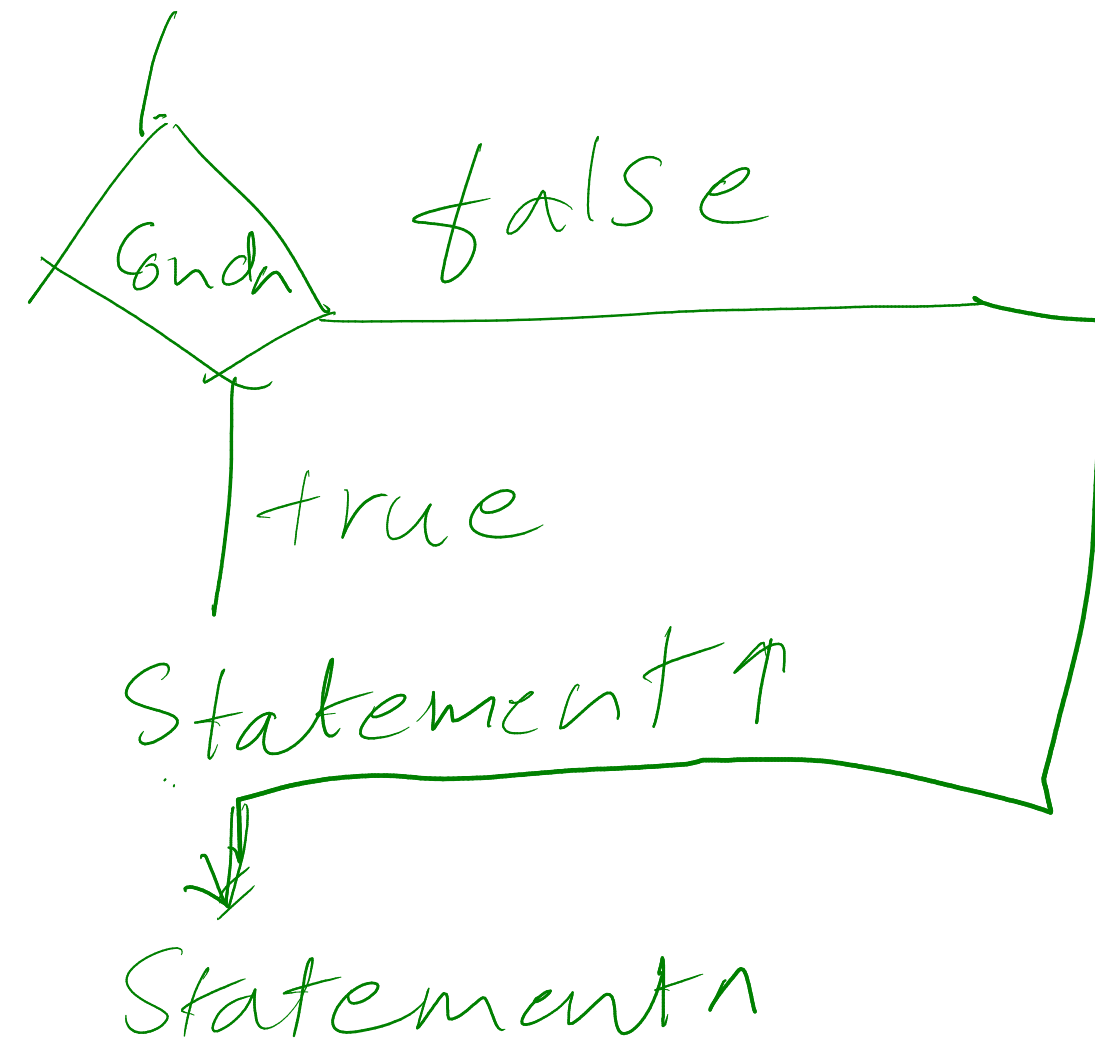
>>> l4.clear()
>>> l4
[]
>>> l5 = l3
>>> l4 = l3.copy()
>>> l5
['apple', 'banana', 'orange', 'mango']
>>> l4
['apple', 'banana', 'orange', 'mango']
>>> l3
['apple', 'banana', 'orange', 'mango']
>>> l3.append('mango')
>>> len(l3)
5
>>> len(l4)
4
>>> len(l5)
5
>>> l4
['apple', 'banana', 'orange', 'mango']
>>> del l4[2]
>>> l4
['apple', 'banana', 'mango']
>>> del l4
>>> l4
Traceback (most recent call last):
  File "<python-input-133>", line 1, in <module>
    l4
NameError: name 'l4' is not defined. Did you mean: 'l1'?
>>>
  
```

```

>>> v = "vishwas"
>>> type(v)
<class 'str'>
>>> v1 = 'vishwas'
>>> type(v1)
<class 'str'>
>>> v2 = '''vishwas'''
>>> type(v2)
<class 'str'>
>>> num = input("Enter a number: ")
Enter a number: 123
>>> num
'123'
>>> type(num)
<class 'str'>
>>> num = float(input("Enter a number: "))
Enter a number: 123
>>> num
123.0
>>> type(num)
<class 'float'>
>>>
  
```


for loop, while loop()

if condⁿ:
---- Statement 1
Statement n



If Condⁿ:

--- S1

--- S2

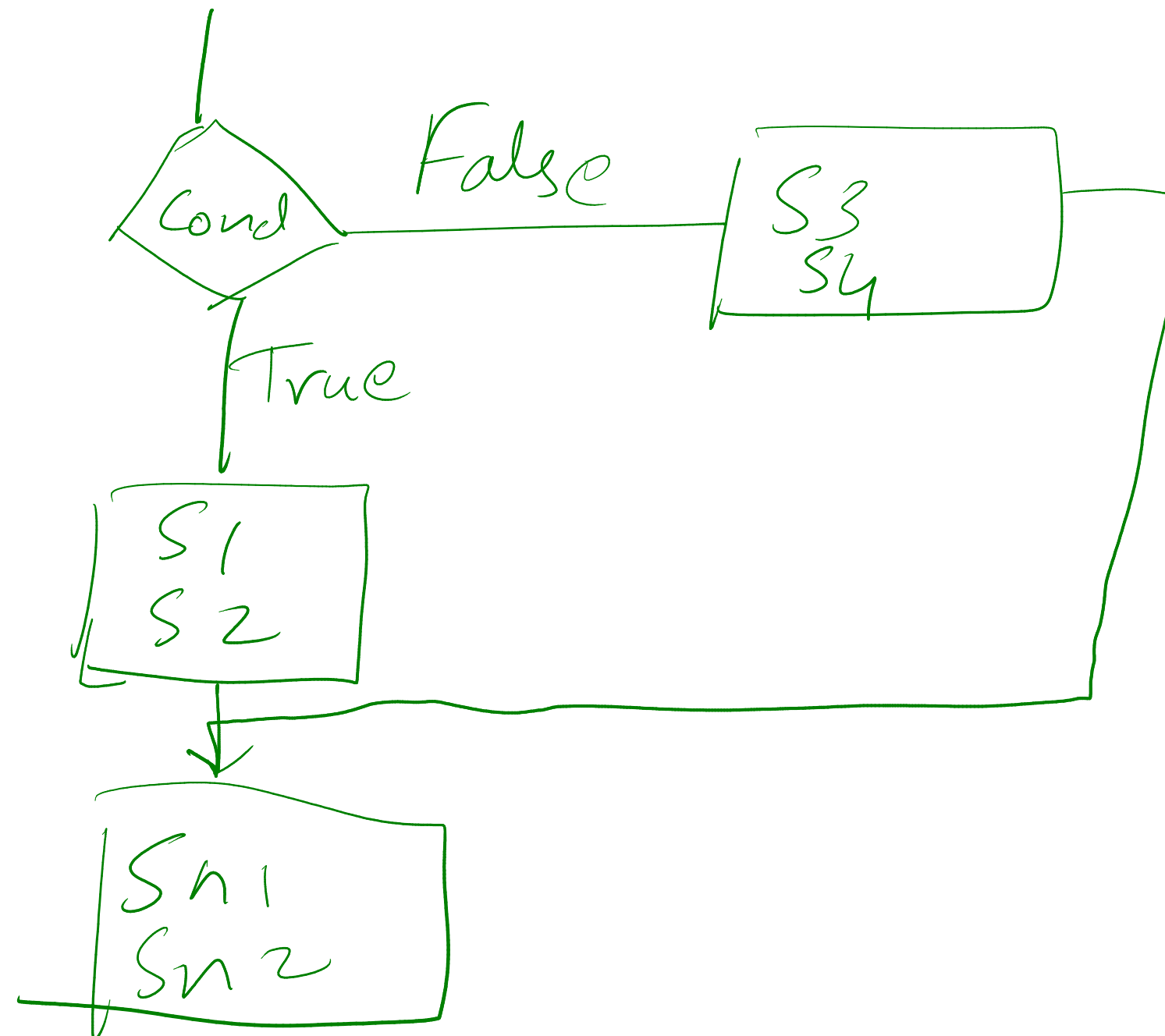
else:

--- S3

--- S4

Sn1

Sn2



if condn1:

---- S1

---- S2

elif condn2:

-- S3

--- S4

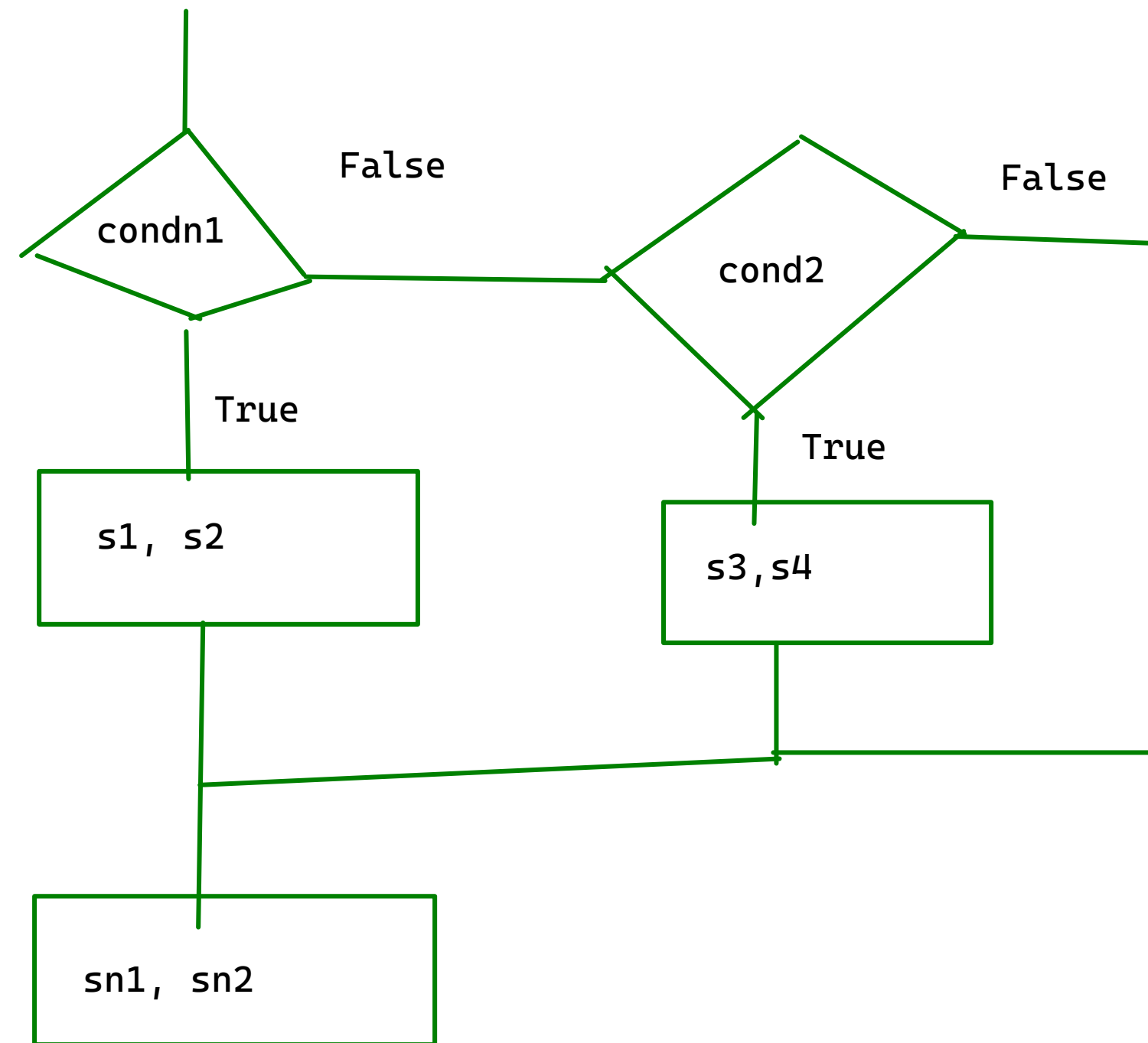
else:

--- S5

--- S6

Sn1

Sn2




```
x = 100
```

What is the output?

```
if x > 50:
    print("Xgt50")
elif x > 70:
    print("Xgt70")
elif x == 100:
    print("Xeq100")
else:
    print("Xlt50")
    print("Completed Successfully")
```

write a IPv4 Address Validator

192.168.1.1

1. It has exactly 4 Fields
2. It has numbers in each field
3. Each Field Varies b/n 0 and 255 only
4. Each field seperated by a dot(.)

1. Step 1 : take input
2. Step 2: Break the string
split('.')
3. type

Dictionaries:

Key Value Pairs

Key must be unique

```
>>> d1 = {}
>>> type(d1)
<class 'dict'>
>>> d2 = {23:"apple",45:"mango",36:"grapes"}
>>> d2
{23: 'apple', 45: 'mango', 36: 'grapes'}
>>> d2[23]
'apple'
>>> d2[45]
'mango'
>>> d2[36]
'grapes'
>>> d3 = {23.45:"Apple",67.98:"Grapes"}
>>> d3
{23.45: 'Apple', 67.98: 'Grapes'}
>>> dir(dict)
['__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__',
 '__getitem__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__ior__', '__iter__', '__le__', '__len__', '__lt__', '__ne__', '__new__',
 '__or__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__ror__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook',
 'clear', 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']
>>> d2[72]
Traceback (most recent call last):
  File "<python-input-261>", line 1, in <module>
    d2[72]
    ~~~~~
KeyError: 72
>>> help(dict.get)
Help on method_descriptor:

get(self, key, default=None, /) unbound builtins.dict method
    Return the value for key if key is in the dictionary, else default.

>>> d2.get(72, "NA")
'NA'
>>> d2.get(23, "NA")
'apple'
>>> d2.keys()
dict_keys([23, 45, 36])
>>> d2.values()
dict_values(['apple', 'mango', 'grapes'])
>>> |
```